problemSet2_ex3.R

cyrilgabriele

2026-04-12

```r
library(quadprog)     # Load quadprog package
options(scipen = 999)

print_heading <- function(title) {
  cat("\n", title, "\n", sep = "")
}

print_metric_table <- function(labels, values, value_name, digits = 4) {
  out <- data.frame(
    Asset = labels,
    Value = round(as.numeric(values), digits),
    row.names = NULL,
    check.names = FALSE
  )
  names(out)[2] <- value_name
  print(out, row.names = FALSE)
}

print_weight_table <- function(labels, weights, digits = 4) {
  out <- data.frame(
    Asset = labels,
    Weight = round(as.numeric(weights), digits),
    row.names = NULL
  )
  print(out, row.names = FALSE)
}

# Import & clean data (same as Ex. 1)
#-------------------------------------------------------------
# Save time series data from .csv file into a data frame
data <- read.csv2("Asset_Classes.csv", header = TRUE, stringsAsFactors = FALSE)

# Convert the first column of the data frame to date format
data$Date <- as.Date(data$Date, "%d.%m.%Y")

# Convert remaining columns to numerical format
for(i in 2:length(data[1,])){data[,i] <- as.numeric(data[,i])}

# Display head and check structure (str) of the data frame
print_heading("Exercise 3 - Data Preview")
```

```
##
```

```
## Exercise 3 - Data Preview
```

```r
print(head(data))
```

```
##         Date  Stocks Treasuries CorpBonds RealEstate     Rf
## 1 1993-01-29  0.0103     0.0112    0.0233     0.0050 0.0072
## 2 1993-02-26  0.0818     0.0256    0.0230     0.0067 0.0072
## 3 1993-03-31  0.0133     0.0066    0.0035     0.0093 0.0069
## 4 1993-04-30 -0.0209     0.0002    0.0077     0.0044 0.0065
## 5 1993-05-31  0.0108     0.0043    0.0012     0.0032 0.0065
## 6 1993-06-30  0.0453     0.0126    0.0243     0.0051 0.0065
```

```r
print_heading("Exercise 3 - Data Structure")
```

```
##
## Exercise 3 - Data Structure
```

```r
str(data)
```

```
## 'data.frame':    240 obs. of  6 variables:
##  $ Date      : Date, format: "1993-01-29" "1993-02-26" ...
##  $ Stocks    : num  0.0103 0.0818 0.0133 -0.0209 0.0108 0.0453 0.0741 0.0701 -0.0247 0.0578 ...
##  $ Treasuries: num  0.0112 0.0256 0.0066 0.0002 0.0043 0.0126 0.0123 0.0172 0.0103 0.0158 ...
##  $ CorpBonds : num  0.0233 0.023 0.0035 0.0077 0.0012 0.0243 0.0072 0.0249 0.0024 0.005 ...
##  $ RealEstate: num  0.005 0.0067 0.0093 0.0044 0.0032 0.0051 0.0072 0.0107 0.0064 0.0077 ...
##  $ Rf        : num  0.0072 0.0072 0.0069 0.0065 0.0065 0.0065 0.0059 0.0058 0.0057 0.0055 ...
```

```r
# split now the data & use the hint i.e. by using the indices
# January 1993 - December 2002
train_data <- data[1:120, ]
# January 2003 - December 2012
test_data <- data[121:240, ]

#-------------------------------------------------------------
#-------------------------------------------------------------
# Markowitz model

# expected returns, standard deviations, and the covariance matrix of the returns
# Number of risky assets: exclude Date and Rf
N <- ncol(train_data) - 2

# Create vector to store mean returns
MU <- rep(NA, N)

# Fill vector with monthly mean returns for each asset
for(i in 2:(ncol(train_data)-1)) {
  MU[i-1] <- mean(train_data[, i], na.rm = TRUE) * 12
}

# Covariance matrix of risky asset returns only
SIGMA <- cov(na.omit(train_data[, 2:(ncol(train_data)-1)])) * 12
```

```r
# Variances and standard deviations
VAR <- diag(SIGMA)
SD <- sqrt(VAR)
asset_names <- colnames(train_data)[2:(ncol(train_data)-1)]
names(MU) <- asset_names
names(SD) <- asset_names
dimnames(SIGMA) <- list(asset_names, asset_names)

print_heading("Exercise 3a - Annualized Expected Returns")
```

```
##
## Exercise 3a - Annualized Expected Returns
```

```r
print_metric_table(asset_names, MU, "ExpectedReturn", digits = 4)
```

```
##       Asset ExpectedReturn
##      Stocks         0.1282
##  Treasuries         0.0703
##   CorpBonds         0.0763
##  RealEstate         0.0491
```

```r
print_heading("Exercise 3a - Annualized Standard Deviations")
```

```
##
## Exercise 3a - Annualized Standard Deviations
```

```r
print_metric_table(asset_names, SD, "StdDev", digits = 4)
```

```
##       Asset StdDev
##      Stocks 0.2052
##  Treasuries 0.0326
##   CorpBonds 0.0478
##  RealEstate 0.0091
```

```r
print_heading("Exercise 3a - Annualized Covariance Matrix")
```

```
##
## Exercise 3a - Annualized Covariance Matrix
```

```r
print(round(SIGMA, 6))
```

```
##               Stocks Treasuries CorpBonds RealEstate
## Stocks      0.042106  -0.000082  0.000519  -0.000070
## Treasuries -0.000082   0.001061  0.000950   0.000161
## CorpBonds   0.000519   0.000950  0.002283   0.000122
## RealEstate -0.000070   0.000161  0.000122   0.000083
```

```r
#---------------------------
# preparation for the optimization
# Define number and type of constraints
n <- 2 + N   # Total number of constraints (2 basic + N short-sale constraints)
meq <- 2     # First "meq" constraints treated as equalities

# dvec set to zero (we specify the target function in terms of the variance only)
dvec <- rep(0,N)

# Amat includes left-hand-side values of the constraints; see **slide 14**
Amat <- matrix(NA, N, n)     # Create matrix to store the constraints
Amat[,1] <- MU               # (i)   w1*mu1 + w2*mu2 = mup (find minimum variance for a given mean)
Amat[,2] <- rep(1, N)        # (ii)  SUM(w_i) = 1          (budget constraint)
for(i in 1:N){               # (iii) w_i >= 0              (short-sale constraints)
  constraint <- rep(0,N)     # Create vector with N zeros
  constraint[i] <- 1         # Set 1 for the short-sale constraints (w*1), 0 for unconstrained frontier
  Amat[,i+2] <- constraint}

stepsize <- 0.0001                        # Stepsize for vector with fixed mean returns for optimizatio
mu_bar <- seq(min(MU),max(MU),stepsize)   # Fixed mean returns for optimization (first constraint)
#mu_bar <- seq(0,1, stepsize)             # Use this for unconstrained frontier

#-----------------------------------------------------------
# Create objects to store efficient frontier
#-----------------------------------------------------------
w <- matrix(NA, N, length(mu_bar))        # Matrix for weights of efficient portfolios
# w[,1:10]                                # View first 10 columns of matrix w to see structure
mu <- rep(NA,length(mu_bar))              # Vector for means of efficient portfolios
sigma <- rep(NA,length(mu_bar))           # Vector for standard deviations of efficient portfolios
#-----------------------------------------------------------
#### Optimization for Efficient Frontier ####
# bvec = c(mu_bar[i], 1, rep(0,N))
# these contain the constrains from **page 14** of the lecture slides
#-----------------------------------------------------------
for(i in 1:length(mu_bar)) {
  w[,i] <- solve.QP(Dmat = SIGMA,
                    dvec = dvec,
                    Amat = Amat,
                    bvec = c(mu_bar[i], 1, rep(0,N)),
                    meq = meq)$solution
  mu[i] <- t(w[,i]) %*% MU
  sigma[i] <- sqrt(t(w[,i]) %*% SIGMA %*% w[,i])}

#-----------------------------------------------------------^
# Find minimum variance portfolio
#-----------------------------------------------------------
# Look up the position of the smallest standard deviation
# in the vector "sigma". This position index is stored as
# scalar object "min_var"

min_var <- which.min(abs(min(sigma) - sigma))
#Alternative that finds closest value
#-----------------------------------------------------------
```

```r
print_heading("Exercise 3a - Minimum Variance Portfolio")
```

```
##
## Exercise 3a - Minimum Variance Portfolio
```

```r
cat("Index on efficient frontier:", min_var, "\n")
```

```
## Index on efficient frontier: 4
```

```r
cat("Annualized expected return:", round(mu[min_var], 4), "\n")
```

```
## Annualized expected return: 0.0494
```

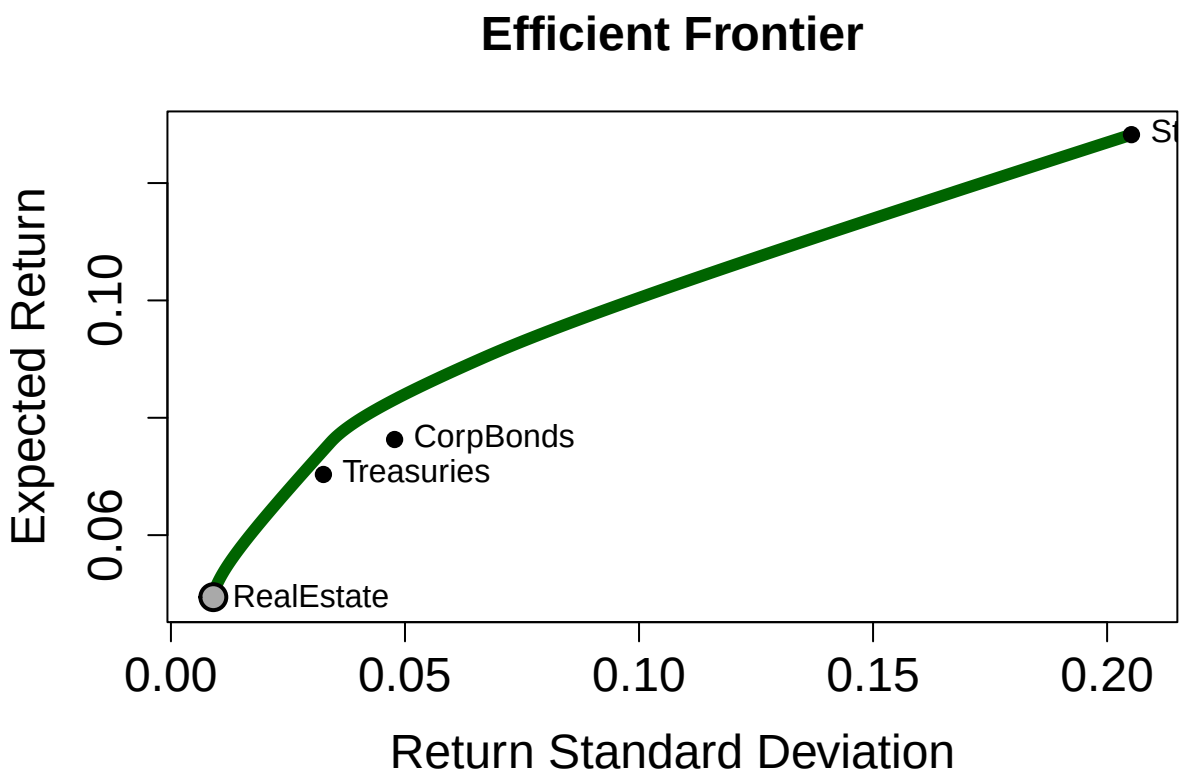
```r
cat("Annualized standard deviation:", round(sigma[min_var], 4), "\n")
```

```
## Annualized standard deviation: 0.0091
```

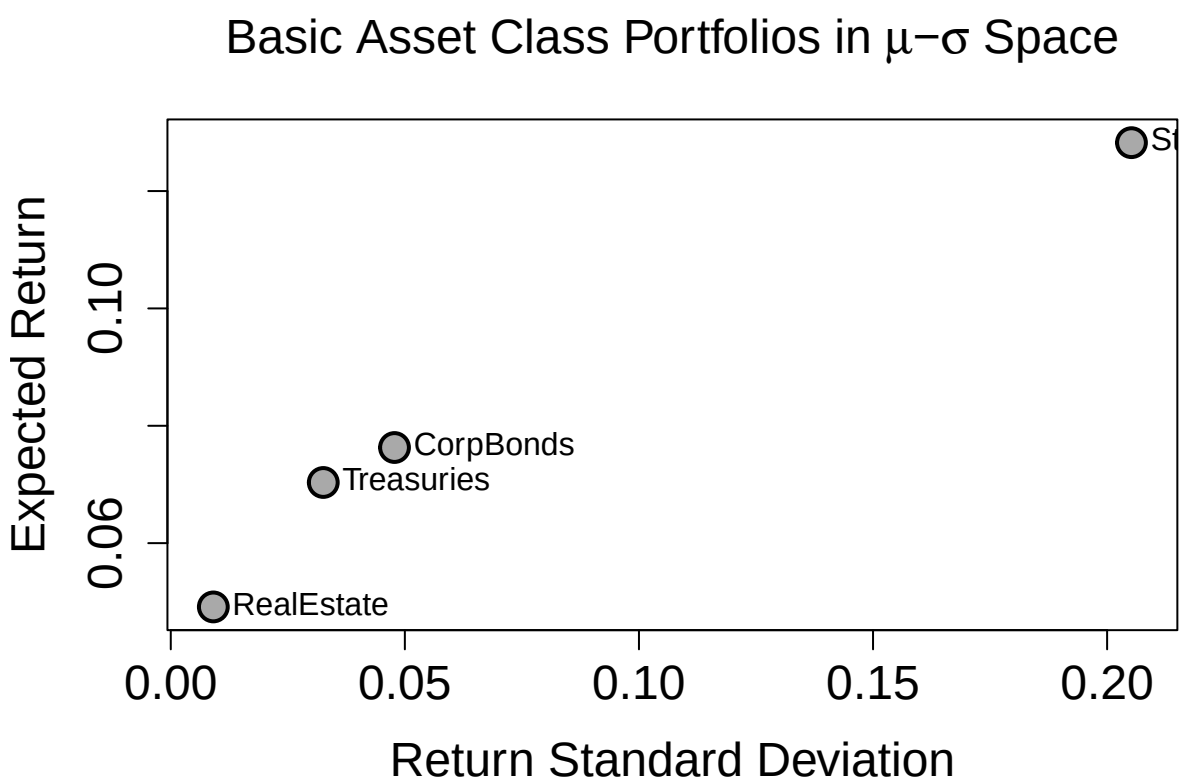
```r
print_weight_table(asset_names, w[, min_var], digits = 4)
```

```
##       Asset Weight
##      Stocks 0.0038
##  Treasuries 0.0000
##   CorpBonds 0.0000
##  RealEstate 0.9962
```

```r
##########################################
# Plot the efficient frontier
##########################################
eff <- mu >= mu[min_var]

x_all <- c(sigma[eff], SD)
y_all <- c(mu[eff], MU)

x_pad <- 0.05 * diff(range(x_all))
y_pad <- 0.05 * diff(range(y_all))

plot(sigma[eff], mu[eff], type = "l",
     lwd = 6,
     xlim = range(x_all) + c(-x_pad, x_pad),
     ylim = range(y_all) + c(-y_pad, y_pad),
     xaxs = "i", yaxs = "i", col = "darkgreen",
     main = "Efficient Frontier",
     xlab = "Return Standard Deviation", ylab = "Expected Return",
     cex.axis = 1.5, cex.lab = 1.5, cex.main = 1.5)

points(SD, MU, pch = 16, cex = 1.2, col = "black")
text(SD, MU, labels = asset_names, pos = 4, cex = 1)

points(sigma[min_var], mu[min_var],
       pch = 21, cex = 1.8, lwd = 2,
       col = "black", bg = "darkgrey")
```




```
#########################################
# Plot the 4 basic asset classes in the mu-sigma space
#########################################
asset_names <- colnames(train_data)[2:(ncol(train_data)-1)]

x_pad <- 0.05 * diff(range(SD))
y_pad <- 0.05 * diff(range(MU))

plot(SD, MU, type = "p",
     pch = 21, cex = 2, lwd = 2,
     col = "black", bg = "darkgrey",
     xlim = range(SD) + c(-x_pad, x_pad),
     ylim = range(MU) + c(-y_pad, y_pad),
     xaxs = "i", yaxs = "i",
     main = expression(paste("Basic Asset Class Portfolios in ", mu, "-", sigma, " Space")),
     xlab = "Return Standard Deviation", ylab = "Expected Return",
     cex.axis = 1.5, cex.lab = 1.5, cex.main = 1.5)

text(SD, MU, labels = asset_names, pos = 4, cex = 1)
```




```r
##########################################
# 1/N Strategy
##########################################
# Monthly return vector of the equally weighted portfolio
ret_1N <- rowMeans(train_data[, 2:(ncol(train_data)-1)])

# Sample mean and sample standard deviation of historical monthly returns
mean_1N_monthly <- mean(ret_1N)
mean_1N_annualized <- mean_1N_monthly * 12

sd_1N_monthly   <- sd(ret_1N)
sd_1N_annualized <- sqrt(12) * sd_1N_monthly

print_heading("Exercise 3b - 1/N Portfolio Summary")
```

```
##
## Exercise 3b - 1/N Portfolio Summary
```

```r
cat("Monthly mean return:", round(mean_1N_monthly, 4), "\n")
```

```
## Monthly mean return: 0.0068
```

```r
cat("Annualized mean return:", round(mean_1N_annualized, 4), "\n")
```

```
## Annualized mean return: 0.081
```

```r
cat("Monthly standard deviation:", round(sd_1N_monthly, 4), "\n")
```

```
## Monthly standard deviation: 0.0159
```

```r
cat("Annualized standard deviation:", round(sd_1N_annualized, 4), "\n")
```

```
## Annualized standard deviation: 0.0552
```

```r
idx_nearest_efficient_portfolio <- which.min(abs(sigma - sd_1N_annualized))
w_eff <- w[, idx_nearest_efficient_portfolio]

print_heading("Exercise 3c - Efficient Portfolio Matching 1/N Risk")
```

```
##
## Exercise 3c - Efficient Portfolio Matching 1/N Risk
```

```r
cat("Index on efficient frontier:", idx_nearest_efficient_portfolio, "\n")
```

```
## Index on efficient frontier: 370
```

```r
cat("Annualized expected return:", round(mu[idx_nearest_efficient_portfolio], 4), "\n")
```

```
## Annualized expected return: 0.086
```

```r
cat("Annualized standard deviation:", round(sigma[idx_nearest_efficient_portfolio], 4), "\n")
```

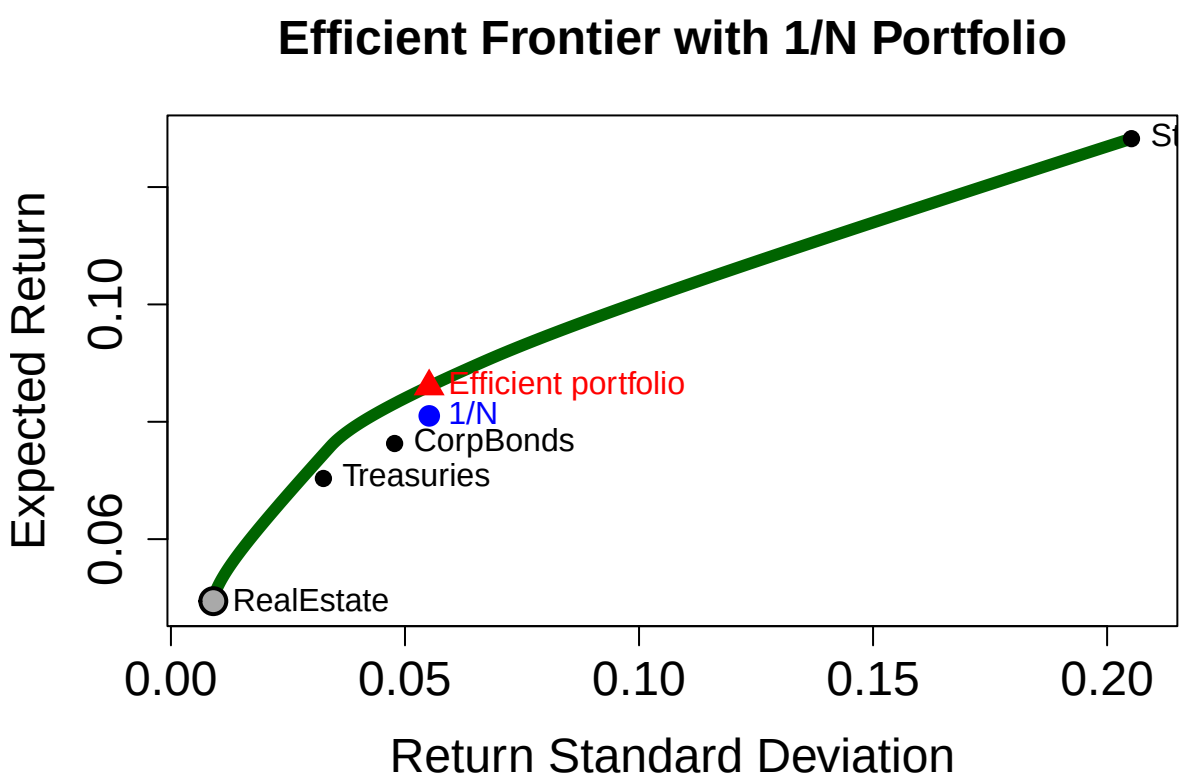
```
## Annualized standard deviation: 0.0552
```

```r
print_weight_table(asset_names, w_eff, digits = 4)
```

```
##       Asset Weight
##      Stocks 0.2118
##  Treasuries 0.2140
##   CorpBonds 0.5742
##  RealEstate 0.0000
```

```r
##########################################
# Replot efficient frontier and add 1/N portfolio
##########################################

asset_names <- colnames(train_data)[2:(ncol(train_data)-1)]
eff <- mu >= mu[min_var]

x_all <- c(sigma[eff], SD, sd_1N_annualized)
y_all <- c(mu[eff], MU, mean_1N_annualized)

x_pad <- 0.05 * diff(range(x_all))
y_pad <- 0.05 * diff(range(y_all))
```

```r
plot(sigma[eff], mu[eff], type = "l",
     lwd = 6,
     xlim = range(x_all) + c(-x_pad, x_pad),
     ylim = range(y_all) + c(-y_pad, y_pad),
     xaxs = "i", yaxs = "i", col = "darkgreen",
     main = "Efficient Frontier with 1/N Portfolio",
     xlab = "Return Standard Deviation", ylab = "Expected Return",
     cex.axis = 1.5, cex.lab = 1.5, cex.main = 1.5)

# Asset class portfolios
points(SD, MU, pch = 16, cex = 1.2, col = "black")
text(SD, MU, labels = asset_names, pos = 4, cex = 1)
# Minimum-variance portfolio
points(sigma[min_var], mu[min_var],
       pch = 21, cex = 1.8, lwd = 2,
       col = "black", bg = "darkgrey")
# --- --- ---
# 1/N portfolio
points(sd_1N_annualized, mean_1N_annualized,
       pch = 16, cex = 1.5, col = "blue")
text(sd_1N_annualized, mean_1N_annualized,
     labels = "1/N", pos = 4, cex = 1, col = "blue")
# --- --- ---
# Efficient portfolio with same standard deviation as 1/N
points(sigma[idx_nearest_efficient_portfolio],
       mu[idx_nearest_efficient_portfolio],
       pch = 17, cex = 1.6, col = "red")
text(sigma[idx_nearest_efficient_portfolio],
     mu[idx_nearest_efficient_portfolio],
     labels = "Efficient portfolio", pos = 4, cex = 1, col = "red")
```




```
#########################################
# d) Out-of-sample Sharpe Ratios
#########################################

# Test-period risky asset returns (exclude Date and Rf)
test_returns <- test_data[, 2:(ncol(test_data)-1)]

# Contemporaneous monthly risk-free rates in the test period
rf_test <- test_data[, ncol(test_data)]   # same as test_data$Rf if the column is named "Rf"

# 1/N portfolio: realized monthly returns in the test period
ret_1N_test <- rowMeans(test_returns)

# Efficient portfolio from part c): realized monthly returns in the test period
ret_eff_test <- as.numeric(as.matrix(test_returns) %*% w_eff)

# Excess returns using the contemporaneous monthly risk-free rate
excess_1N_test  <- ret_1N_test  - rf_test
excess_eff_test <- ret_eff_test - rf_test

# Out-of-sample Sharpe ratios (monthly, as defined in the problem set)
SR_1N_oos  <- mean(excess_1N_test)  / sd(excess_1N_test)
SR_eff_oos <- mean(excess_eff_test) / sd(excess_eff_test)

# Optional: store everything in one data frame for inspection
oos_results <- data.frame(
```

```r
  Date = test_data$Date,
  rf = rf_test,
  ret_1N = ret_1N_test,
  ret_eff = ret_eff_test,
  excess_1N = excess_1N_test,
  excess_eff = excess_eff_test
)

# Print the Sharpe ratios
print_heading("Exercise 3d - Out-of-Sample Sharpe Ratios")
```

```
##
## Exercise 3d - Out-of-Sample Sharpe Ratios
```

```r
sharpe_results <- data.frame(
  Portfolio = c("1/N", "Efficient"),
  MeanExcessReturn = round(c(mean(excess_1N_test), mean(excess_eff_test)), 4),
  StdDevExcessReturn = round(c(sd(excess_1N_test), sd(excess_eff_test)), 4),
  SharpeRatio = round(c(SR_1N_oos, SR_eff_oos), 4),
  row.names = NULL
)
print(sharpe_results, row.names = FALSE)
```

```
##  Portfolio MeanExcessReturn StdDevExcessReturn SharpeRatio
##        1/N           0.0028             0.0144      0.1918
##  Efficient           0.0032             0.0169      0.1867
```